

Chapitre 5: Optimisation

H : la fonction à minimiser ($-H$ pour maximiser)

Mesure de Gibbs: $\pi_\beta(x) = \frac{1}{Z} e^{-\beta H(x)}$

$\mathcal{C} = \{x \mid H(x) = m\}$ avec m : minimum de H .

Exemple: $\mathcal{C}$: l'ensemble des "bons" coloriage.

Proposition: Pour tout x , $\lim_{\beta \rightarrow \infty} \pi_\beta(x) = \frac{1_{\mathcal{C}}(x)}{\text{Card}(\mathcal{C})}$.

Explication de la limite: $\text{Card}(\mathcal{C}) = 15$, donc $\frac{1_{\mathcal{C}}(x)}{\text{Card}(\mathcal{C})} = \begin{cases} \frac{1}{15} & \text{si } x \in \mathcal{C} \\ 0 & \text{si } x \notin \mathcal{C} \end{cases}$

C'est une uniforme discrète sur $\mathcal{C}$.

Démonstration:
$$\begin{aligned} \pi_\beta(x) &= \frac{e^{-\beta H(x)}}{\sum_{z \in \mathcal{C}} e^{-\beta H(z)} + \sum_{z \notin \mathcal{C}} e^{-\beta H(z)}} \\ &= \frac{e^{-\beta H(x)}}{\text{Card}(\mathcal{C}) e^{-\beta m} + \sum_{z \notin \mathcal{C}} e^{-\beta H(z)}} \\ &= \frac{e^{-\beta(H(x)-m)}}{\text{Card}(\mathcal{C}) + \sum_{z \notin \mathcal{C}} e^{-\beta(H(z)-m)}} \end{aligned}$$

$\xrightarrow{\beta \rightarrow \infty} \begin{cases} 1 & \text{si } x \in \mathcal{C} \\ 0 & \text{si } x \notin \mathcal{C} \end{cases}$

$\rightarrow 0$ avec $\beta \rightarrow \infty$
 $\rightarrow 0$ (la somme est finie qui tend vers 0)

Conclusion: Si je simule π_β avec β grand, je simule presque une uniforme discrète sur $\mathcal{C}$, et c'est bon

Problème: Avec β très grand, l'algorithme de Metropolis accepte uniquement les points qui font descendre la fonction H , donc on risque de tomber dans un minimum local.

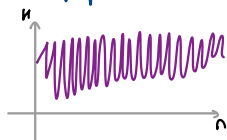
Solution: C'est d'augmenter progressivement le paramètre β lors des étapes de l'algo: "le recuit simulé".

- * H : fonction d'énergie
- * β : Paramètre de température

Théorème: Si $\beta = \beta_0 \log(i)$, où i sont les étapes de l'algorithme, alors l'algorithme va permettre de s'approcher du minimum.

"Tuning" de β_0 :

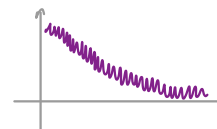
Si β_0 est trop petit:



Si β_0 est trop grand:



β_0 convenable:



29

Problème du voyageur de commerce (TSP problem: Traveling Salesman Problem)

Problème: Partir d'une ville, en visiter N (une seule fois), et revenir à la ville de départ, en minimisant la distance parcourue.

Modélisation: Je définis G un vecteur de taille N : $G[i] = j$ signifie que je visite la ville j en i -ème position.

$G \in S_N$: Permutations à N éléments

$\text{card}(S_N) = N!$ problème de taille exponentielle

$H(G)$: Fonction qui calcule le trajet par l'ordre de passage.

Nécessite une matrice de distance entre les villes V .

Transition: $G: (12 \quad 24 \quad \textcircled{7} \quad 38 \quad \dots \quad \textcircled{3} \quad 25 \quad 17 \quad 9)$

$G_{\text{new}}: (\dots \dots \dots \quad \swarrow \quad \searrow \quad \dots \dots \dots)$